Guía de estudio previa · SID2 · Evaluación 1

Arquitectura de SGBD, OLTP/OLAP, administración y optimización en Oracle

Propósito de esta guía
Esta guía está pensada para estudiar antes de la evaluación. Está organizada para repasar teoría, ubicar rápidamente conceptos clave y practicar en Oracle SQL Developer con ejemplos muy parecidos a los laboratorios del curso.

1. Cómo usar esta guía

- Lee primero el mapa general del parcial para ubicar qué temas pertenecen a teoría y cuáles a práctica.
- En teoría, concéntrate en distinguir conceptos que suelen confundirse: ANSI/SPARC vs arquitectura del motor, OLTP vs OLAP, usuario vs rol, vista vs sinónimo, join type vs join method, explain plan vs plan realmente ejecutado.
- En práctica, no memorices solo comandos aislados: entiende para qué se usa cada sentencia, con qué usuario se ejecuta, qué privilegio requiere y cómo verificar si funcionó.
- Usa las plantillas SQL de la parte práctica para ensayar: crear usuario, asignar quota, crear rol, otorgar permisos, crear vista, crear sinónimo, revisar plan de ejecución y reescribir consultas.

2. Mapa rápido del parcial

Bloque	Qué te pueden preguntar	Qué debes dominar
Teórico	Selección múltiple, abiertas e interpretación de imágenes	Arquitectura de SGBD, ANSI/SPARC, Oracle vs PostgreSQL, OLTP vs OLAP, modelo dimensional, atributos de calidad, joins, optimizer, índices, vistas, sinónimos y seguridad
Práctico	Resolución en Oracle SQL Developer	Usuarios, roles, permisos, tablespaces, cuotas, vistas, sinónimos, corrección de scripts, consultas con joins, explain plan, lectura del plan y optimización

PARTE I · GUÍA TEÓRICA

3. Base conceptual: qué es un sistema de base de datos y qué hace un SGBD

Un sistema de base de datos no es solo “las tablas”. Incluye hardware, software, datos y personas. El SGBD es el software que permite construir, usar, mantener y reorganizar la base de datos. Debe manejar persistencia, lenguaje de alto nivel, control de acceso, integridad, concurrencia, recuperación y propiedades ACID.

- Actores típicos: DBA, diseñador de base de datos, programadores de aplicaciones y usuarios finales.

- Capacidades esperadas del SGBD: independencia de datos, acceso eficiente, integridad, seguridad, concurrencia, recuperación y soporte de transacciones.
- Pregunta clásica: “¿esquema o instancia?” → esquema es la definición estructural; instancia es el contenido real en un momento dado.

Idea clave para una abierta
El curso insiste en que un SGBD no solo almacena datos: también controla cómo se accede a ellos, cómo se protegen, cómo se recuperan tras fallos y cómo se optimiza su consulta.

4. Arquitectura ANSI/SPARC: tres niveles e independencia de datos

ANSI/SPARC es un modelo de abstracción de datos. No describe procesos internos del motor; describe niveles de representación para separar la visión del usuario de los detalles físicos.

Nivel	Qué describe	Palabras clave del parcial
Externo	Cómo usuarios o aplicaciones ven los datos	Vistas, formatos, confidencialidad, representaciones distintas
Conceptual	Qué datos existen y cómo se relacionan	Esquema lógico global, integridad, coherencia, reglas
Interno	Cómo se almacenan físicamente	Índices, bytes, páginas, reserva de espacio, compresión, estructuras de acceso

La independencia de datos es la razón de ser de esta arquitectura.

- Independencia física: puedo cambiar almacenamiento, índices o estructura física sin cambiar las vistas externas.
- Independencia lógica: puedo ajustar el esquema conceptual sin obligar a reescribir todas las aplicaciones.

No confundas esto
ANSI/SPARC define niveles. La arquitectura interna del SGBD define componentes. Una cosa es el modelo de abstracción; otra, cómo Oracle o PostgreSQL implementan el motor que hace posible esa abstracción.

5. Componentes de un SGBD y responsabilidades internas

En clase se trabaja mucho la idea de componentes y funciones. Si te ponen una función del SGBD, piensa: ¿qué componente es el principal y con cuáles interactúa?

Función	Componente principal	Qué hace realmente
Verificar permisos antes de ejecutar	Security / control de acceso	Valida usuario, privilegios y objeto solicitado
Interpretar CREATE TABLE	Parser + procesador de consultas + catálogo	Valida sintaxis, semántica y registra metadatos
Optimizar un SELECT con varios joins	Optimizer	Escoge access paths, join order y join methods
Ejecutar el plan	SQL execution engine	Corre cada operación del plan y devuelve filas
Mantener atomicidad ante rollback	Transaction manager	Usa undo / control transaccional

Decidir páginas en memoria	Buffer manager / memory manager	Administra caché y reemplazo
Acceder físicamente a registros	Storage manager	Lee bloques, índices, segmentos y archivos
Usar estadísticas para elegir plan	Cost-based optimizer	Estima cardinalidades y costos

6. Oracle vs PostgreSQL: comparación que sí conviene memorizar

El parcial puede mezclar preguntas conceptuales y comparativas. No memorices nombres sueltos; memoriza las diferencias arquitectónicas importantes.

Aspecto	Oracle	PostgreSQL
Estilo arquitectónico	Más integrado y centralizado	Más modular y process-per-user
Memoria	SGA compartida + PGA por proceso	shared_buffers, work_mem y memoria por backend
Concurrencia / MVCC	MVCC apoyado en undo segments	MVCC con versiones de tuplas
Recuperación	Redo logs + undo + checkpoints + background processes	WAL + checkpoints
Escalabilidad / HA	RAC nativo y ecosistema Oracle	Replicación y herramientas externas
Cliente-servidor	Server processes y background processes	postmaster/postgres y un proceso por conexión

En Oracle una instancia es el conjunto de memoria y procesos que acceden a la base de datos. No es lo mismo “instancia” que “base de datos”.

- SGA: memoria compartida del estado general de la instancia.
- PGA: memoria privada de cada proceso.
- Procesos de Oracle que debes reconocer: SMON, PMON, DBW/DBWR, LGWR, CKPT, ARCH.
- En PostgreSQL recuerda la idea de postmaster, backends, shared buffers y storage manager.

7. ACID, concurrencia, logging y recuperación

Aunque la práctica de tu evaluación se centra en administración y optimización, la parte teórica puede usar arquitectura para preguntar por transacciones, consistencia y confiabilidad.

Propiedad	Qué garantiza	Qué debes saber explicar
Atomicidad	Todo o nada	Si algo falla, rollback; no quedan cambios a medias
Consistencia	La BD pasa de un estado válido a otro válido	Se respetan reglas, restricciones y lógica del negocio
Aislamiento	Una transacción no ve estados intermedios de otra	Evita interferencias; se relaciona con concurrencia
Durabilidad	Lo confirmado persiste	Commit debe sobrevivir a fallos razonables

Relaciona además los mecanismos con los atributos de calidad: consistencia y confiabilidad se sostienen con control de concurrencia, redo/WAL, checkpoints, recuperación y manejo correcto del commit/rollback.

- Oracle: redo logs, undo, checkpoints y procesos de recuperación.

- PostgreSQL: WAL, checkpoints y MVCC.
- La concurrencia impacta rendimiento y consistencia al mismo tiempo.

8. OLTP vs OLAP

Este contraste es central en la evaluación porque corresponde a RA2. Te pueden preguntar por diferencias, por casos de uso o por imágenes de modelos.

Aspecto	OLTP	OLAP
Propósito	Operación diaria del negocio	Análisis y toma de decisiones
Consultas	CRUD y búsquedas simples	Agregaciones, tendencias, históricos, comparaciones
Modelo	Normalizado	Dimensional / denormalizado
Carga	Muchísimas transacciones pequeñas	Menos consultas pero más pesadas
Tiempo de respuesta	Muy bajo, casi inmediato	Puede tolerar más tiempo por consulta compleja
Datos	Actuales y operativos	Históricos e integrados
Usuarios	Operativos	Analistas, gerentes, directivos

Regla de oro
OLTP prioriza integridad y velocidad transaccional. OLAP prioriza análisis, agregación y soporte a decisiones. El modelo de datos y la forma de optimizar no son iguales.

- OLTP soporta business logic: ventas, reservas, banca, e-commerce.
- OLAP soporta business intelligence: analítica de ventas, desempeño, marketing, supply chain.
- Del mundo OLTP al OLAP aparece la ingesta: recolectar, integrar, transformar y cargar datos para análisis.

9. Data warehouse, hechos, dimensiones, medidas y jerarquías

Un Data Warehouse es un repositorio centralizado de datos integrados, estructurados e históricos diseñado para análisis. Cuando te pregunten por modelo estrella o copo de nieve, piensa en cómo se organiza ese análisis.

Concepto	Qué es	Cómo reconocerlo
Tabla de hechos	Tabla central del análisis	Contiene claves foráneas a dimensiones y medidas numéricas
Dimensión	Describe el contexto del hecho	Tiempo, producto, cliente, tienda, vendedor
Medida	Valor que se agrega	Ventas, cantidad, costo, margen
Jerarquía	Niveles de agregación	Día → mes → trimestre → año; ciudad → región → país
Grano	Nivel de detalle de la tabla de hechos	Una fila por venta, por línea de venta, por clic, etc.

El grano es crítico. Antes de responder cualquier consulta sobre una fact table, define exactamente qué representa una fila. Si no sabes el grano, es muy fácil agrupar mal o duplicar valores.

Modelo	Ventaja	Desventaja
Estrella	Más simple y rápido para consultar	Puede repetir datos en dimensiones
Copo de nieve	Reduce redundancia en dimensiones	Más joins y más complejidad

- El modelo estrella favorece lectura y analítica rápida.
- El copo de nieve normaliza dimensiones, pero puede complicar la consulta.
- En OLAP se aceptan estructuras más desnormalizadas porque el objetivo no es el CRUD intensivo sino el análisis.

10. Atributos de calidad en bases de datos

En la Unidad 3 el curso enfatiza cinco atributos de calidad para datos y SGBDR: integridad, seguridad, consistencia, eficiencia y disponibilidad. También aparecen otros atributos como exactitud, completitud, actualidad, usabilidad y universalidad.

Atributo	Idea corta	Cómo lo explicas en una abierta
Integridad	Datos correctos y válidos	Se sostiene con restricciones, claves, tipos y reglas de negocio
Consistencia	Sin contradicciones	Mismo significado y valores coherentes entre tablas y transacciones
Seguridad	Protección contra accesos no autorizados	Usuarios, roles, privilegios, vistas, principio de mínimo privilegio
Eficiencia / rendimiento	Buenas respuestas con buen uso de recursos	Consultas bien escritas, índices, buen plan, buena memoria
Disponibilidad	Datos accesibles cuando se necesitan	Recuperación, logging, respaldo, alta disponibilidad, continuidad

Muy importante: no confundas integridad con consistencia. Integridad apunta al cumplimiento de restricciones y validez de los datos. Consistencia apunta a ausencia de contradicciones y mantenimiento del estado correcto antes y después de operaciones.

11. Restricciones e integridad

- Restricciones de dominio: tipo y rango válido de un atributo.
- Integridad referencial: una FK debe referenciar un valor existente en la PK o UK padre, salvo nulos si aplica.
- Asertos / condiciones de negocio: reglas que la base debe satisfacer siempre.
- PK: identifica de forma única y no acepta nulos.
- UK: garantiza unicidad, pero no es necesariamente la PK.
- FK: conecta tablas y preserva consistencia referencial.
- CHECK: valida condiciones como rangos o relaciones simples entre columnas.

12. Usuarios, roles, privilegios, tablespaces y cuotas en Oracle

Esta parte es fuertísima para la práctica, pero también puede salir en teoría con justificaciones. Debes poder explicar qué se hace con SYSTEM/SYS, qué se hace con usuarios normales y por qué.

Concepto	Qué significa	Ejemplo típico del curso
Usuario	Cuenta que puede conectarse	OT, CONSULTA, tk1, guest_user, opti
Rol	Agrupación de privilegios	tiktoky_reader
Privilegio de sistema	Autoriza acciones sobre el sistema	CREATE SESSION, CREATE TABLE, CREATE USER
Privilegio sobre objeto	Autoriza acciones sobre un objeto específico	GRANT SELECT ON ot.regions TO consulta
Tablespace	Contenedor lógico de almacenamiento	DATOS, USERS, TBS_INDICES
Quota	Espacio autorizado al usuario en un tablespace	QUOTA 50M ON USERS o UNLIMITED

Ojo con Oracle moderno
En Oracle 12c en adelante, CONNECT y RESOURCE están obsoletos para este tipo de prácticas. En laboratorios del curso se prefiere otorgar privilegios explícitos como CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE, etc.

- SYSTEM o SYS crea usuarios, tablespaces y privilegios elevados.
- El usuario dueño de un objeto puede otorgar privilegios sobre ese objeto, si corresponde.
- Tener CREATE VIEW no implica poder consultar cualquier tabla: también necesitas privilegios sobre las tablas base.
- Quota no es un privilegio de conexión: es el permiso de ocupar espacio en un tablespace.
- Un rol simplifica administración: das privilegios al rol y luego asignas el rol a varios usuarios.

13. Vistas y sinónimos: diferencias que siempre preguntan

Objeto	Qué es	Qué sí hace	Qué no hace
Vista	Consulta almacenada	Oculta complejidad, restringe columnas/filas, puede servir de capa de seguridad	No almacena datos por sí misma (salvo materializadas, que no es el caso aquí)
Sinónimo	Alias de otro objeto	Permite referenciar un objeto sin escribir su esquema o nombre largo	No concede privilegios ni reemplaza GRANT

Una vista normalmente refleja los cambios de las tablas base porque se evalúa sobre ellas. Algunas vistas permiten DML, pero depende de la definición. Si la vista es simple y actualizable, un UPDATE puede propagarse a la tabla base. Si tiene agregaciones, DISTINCT, GROUP BY, set operators o combinaciones complejas, usualmente no será actualizable.

- Dar acceso a una vista en vez de a la tabla base ayuda a aplicar mínimo privilegio.
- Un sinónimo facilita uso, no seguridad. Primero debe existir el privilegio real.
- Puede haber sinónimos privados y públicos. En laboratorio suele interesar el privado.

14. Joins: tipos de join, joins en Oracle y errores típicos

Este tema merece una sección larga porque te lo pidieron explícitamente y además es muy preguntable tanto en teoría como en práctica.

14.1 Join type no es lo mismo que join method

- Join type = resultado lógico: inner, left outer, right outer, full outer, cross, self.
- Join method = algoritmo físico en el plan: nested loops, hash join, merge join.
- En el plan puedes ver HASH JOIN o NESTED LOOPS; eso no te dice por sí solo si el join era left o inner, salvo cuando el plan lo indique con palabras como OUTER, SEMI o ANTI.

14.2 Tipos lógicos de join

Tipo	Qué devuelve	Cuándo usarlo
INNER JOIN	Solo filas que cumplen la condición	Cuando solo interesan coincidencias
LEFT OUTER JOIN	Todas las filas de la izquierda y coincidencias de la derecha	Cuando quieres conservar el lado izquierdo aunque no haya match
RIGHT OUTER JOIN	Todas las filas de la derecha y coincidencias de la izquierda	Mismo concepto, pero preservando la derecha
FULL OUTER JOIN	Coincidencias y no coincidencias de ambos lados	Cuando quieres ver todo, con nulos donde falte match
CROSS JOIN / cartesiano	Cada fila con cada fila	Solo si realmente se necesita; casi siempre es un error
SELF JOIN	Una tabla unida consigo misma	Jerarquías o relaciones dentro de la misma tabla

14.3 Sintaxis ANSI recomendada

```
SELECT e.employee_id, e.last_name, d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id;
```

```
SELECT c.customer_id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id;
```

```
SELECT p.product_id, s.sale_id
FROM products p
FULL OUTER JOIN sales s
  ON s.product_id = p.product_id;
```

14.4 Sintaxis antigua de Oracle con (+)

Aunque hoy se prefiere ANSI JOIN, todavía puede aparecer en material teórico, capturas o preguntas. El (+) se pone en el lado opcional de la condición outer join.

```
-- Equivale a LEFT JOIN de employees hacia departments
SELECT e.last_name, d.department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id(+);
```

Si ves (+), piensa: “la tabla con (+) es la que puede no tener match”. Esta sintaxis es más propensa a errores y menos legible.

14.5 Semijoin y antijoin

Aunque en SQL normalmente escribes EXISTS, IN, NOT EXISTS o NOT IN, el optimizador puede transformar eso internamente a SEMI o ANTI joins.

- Semijoin: devuelve fila del primer conjunto si existe coincidencia en el segundo; en planes puedes ver NESTED LOOPS SEMI o HASH JOIN SEMI.
- Antijoin: devuelve fila del primer conjunto si no existe coincidencia; en planes puedes ver HASH JOIN ANTI o NESTED LOOPS ANTI.

```
SELECT department_id, department_name
FROM departments d
WHERE EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.department_id = d.department_id
);
```

```
SELECT department_id, department_name
FROM departments d
WHERE NOT EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.department_id = d.department_id
);
```

14.6 Errores típicos con joins

- Olvidar una condición de join y producir un producto cartesiano.
- Filtrar en WHERE una tabla outer join de forma que conviertes el outer join en inner join sin querer.
- Hacer joins innecesarios que no aportan columnas ni filtros.
- Unir tablas con columnas de tipos incompatibles o con funciones que impiden buen uso de índices.
- No pensar el grano: en fact tables, un join mal hecho duplica medidas.

15. Join methods en el plan de ejecución

Cuando Oracle ya decidió “qué tablas unir”, todavía debe decidir “cómo unirlas”. Ahí aparecen los join methods.

Join method	Cuándo suele ser razonable	Qué pistas verás
NESTED LOOPS	Pocas filas en el outer y buen acceso a la tabla interna, a menudo por índice	Suele aparecer junto con INDEX RANGE/UNIQUE SCAN
HASH JOIN	Grandes volúmenes y equijoins	Oracle construye hash sobre un lado y sondea con el otro
MERGE JOIN	Entradas ordenadas o cuando ordenar es rentable	Puede aparecer con SORT JOIN en uno o ambos lados

Regla mental útil: nested loops suele ser fuerte cuando recuperas poco y tienes buen índice; hash join suele ser fuerte cuando hay bastante volumen; merge join aparece menos en ejercicios básicos, pero debes reconocerlo.

16. Optimizador, estadísticas y decisiones de plan

Oracle usa optimizador basado en costo. Evalúa access paths, join order, join methods y otras transformaciones. Para eso necesita estadísticas razonables.

- Si las estadísticas están ausentes o desactualizadas, el plan puede ser malo.
- El optimizer intenta reducir filas lo más temprano posible.
- También puede reescribir internamente la consulta: view merging, subquery unnesting, semijoin, antijoin, etc.
- El plan elegido es el de menor costo estimado entre los planes generados.

Muy importante para una abierta
Un mal rendimiento no siempre significa “falta un índice”. También puede deberse a SQL mal diseñado, joins innecesarios, funciones sobre columnas filtradas, estadísticas pobres o un plan subóptimo.

17. Cómo ver, leer y entender un plan de ejecución

Esta es una de las partes más importantes de tu guía. No basta con saber sacar el plan: debes saber interpretarlo.

17.1 Cómo generarlo

```
EXPLAIN PLAN FOR
SELECT *
FROM students
WHERE last_name = 'Smith';

SELECT *
FROM TABLE(DBMS_XPLAN.DISPLAY);
```

En este flujo, EXPLAIN PLAN genera un plan estimado. DBMS_XPLAN.DISPLAY lo muestra. Si el curso llega a pedir plan del cursor real, la idea cambia: debes ejecutar primero la consulta y luego revisar DISPLAY_CURSOR, normalmente con privilegios adicionales.

17.2 Columnas que debes mirar siempre

Columna	Qué significa	Cómo la interpretas
Operation	Operación física/lógica	TABLE ACCESS FULL, INDEX RANGE SCAN, HASH JOIN, etc.
Name	Objeto involucrado	Tabla, índice, vista o alias interno
Rows	Filas estimadas	No son las reales; son estimaciones del optimizador
Bytes	Volumen estimado	Te da idea del tamaño del flujo de datos
Cost	Costo estimado	No es tiempo real; sirve para comparar planes
Time	Tiempo estimado	Orientativo, no exacto
Predicate Information	Condiciones aplicadas	Distingue access de filter

17.3 Cómo se lee

- Lee el plan como árbol de row sources: los hijos alimentan al padre.
- En formato tabular, normalmente se entiende mejor de abajo hacia arriba, identificando qué pasos obtienen datos y cuáles operan sobre esos flujos.

- La indentación importa: las sangrías muestran jerarquía entre operaciones.
- Los pasos de acceso físico suelen ser TABLE ACCESS FULL, INDEX RANGE SCAN, INDEX UNIQUE SCAN, TABLE ACCESS BY INDEX ROWID.
- Los pasos de combinación o procesamiento suelen ser HASH JOIN, NESTED LOOPS, SORT GROUP BY, SORT ORDER BY, BITMAP AND, etc.

17.4 access vs filter

En Predicate Information, Oracle puede mostrar access y filter. Esta diferencia importa mucho.

- access: la condición sirve para entrar al objeto por un camino más selectivo. Ejemplo: usar un índice por dni.
- filter: la condición se aplica después de recuperar filas; suele ser menos eficiente que usarla como access.

```
Predicate Information:
1 - access("DNI"='78901234')
2 - filter(UPPER("COUNTRY")='SPAIN')
```

Interpretación: la primera ayuda a localizar; la segunda puede implicar que Oracle trajo filas y luego filtró, especialmente si no existe índice basado en función.

17.5 Operaciones comunes que debes reconocer

Operación	Lectura rápida
TABLE ACCESS FULL	Oracle lee toda la tabla. No siempre es malo; depende del tamaño y selectividad.
INDEX UNIQUE SCAN	Búsqueda por clave única; suele ser muy selectiva.
INDEX RANGE SCAN	Búsqueda por rango o prefijo; común en B-tree.
TABLE ACCESS BY INDEX ROWID	Oracle primero encontró rowids en el índice y luego fue a la tabla por esas filas.
INDEX FAST FULL SCAN	Lee gran parte o todo el índice como estructura más liviana que la tabla.
BITMAP INDEX / BITMAP AND	Uso o combinación de índices bitmap.
NESTED LOOPS	Join iterativo, muy compatible con índices en la tabla interna.
HASH JOIN	Join por hashing, muy común en grandes volúmenes.
MERGE JOIN	Join sobre entradas ordenadas.
SORT GROUP BY	Ordena/agrupa para agregación.
SORT ORDER BY	Ordena el resultado final.

17.6 Qué preguntas debes hacerte al ver un plan

1. ¿Está leyendo una tabla completa cuando yo esperaría uso de índice?
2. ¿El join method tiene sentido para el volumen de datos?
3. ¿Hay funciones en WHERE que impidan acceso eficiente?
4. ¿Hay joins o columnas seleccionadas que no aportan al resultado?
5. ¿El plan puede resolverse solo con índice o necesita saltar a la tabla por ROWID?
6. ¿El costo, rows o bytes bajan después de reescribir?

18. Índices: B-tree, bitmap, compuestos y ROWID

Los índices son un tema central de la práctica. No aprendas “índice = siempre mejor”. Aprende en qué contexto ayudan.

Tipo	Cuándo sirve	Cuándo no
B-tree	Búsquedas exactas, rangos, prefijos, alta cardinalidad	No siempre ayuda si la consulta trae gran parte de la tabla
Bitmap	Columnas de baja cardinalidad y entornos analíticos	No es ideal para alta concurrencia transaccional
Compuesto	Consultas por varias columnas relacionadas	Su utilidad depende del orden de columnas

18.1 Índice compuesto y regla del prefijo izquierdo

Si creas un índice sobre (last_name, first_name), Oracle puede usarlo muy bien para filtrar por last_name o por last_name + first_name. Pero no necesariamente para filtrar solo por first_name.

```
CREATE INDEX idx_students_last_first
ON students(last_name, first_name)
TABLESPACE tbs_indices;
```

- WHERE last_name = ... → sí puede aprovechar el prefijo.
- WHERE last_name = ... AND first_name = ... → muy buen caso.
- WHERE first_name = ... → no necesariamente aprovecha el índice compuesto.
- WHERE last_name LIKE SMI% → puede usar rango sobre el prefijo.

18.2 ¿Qué es ROWID y por qué aparece en el plan?

ROWID identifica físicamente la ubicación de una fila. En muchos planes Oracle primero consulta un índice, obtiene los ROWID de las filas candidatas y luego va a la tabla usando TABLE ACCESS BY INDEX ROWID.

- Acceder al índice no es lo mismo que acceder a la fila de la tabla.
- Si todas las columnas requeridas están en el índice, a veces Oracle puede resolver la consulta sin visitar la tabla.
- Si necesitas una columna no incluida, Oracle debe ir a la tabla por ROWID.

18.3 Índices bitmap

Un bitmap index suele ser ideal cuando la columna tiene pocos valores distintos, por ejemplo gender o city en análisis masivos. Oracle puede combinar varios bitmaps con operaciones como BITMAP AND.

```
CREATE BITMAP INDEX idx_students_gender
ON students(gender)
TABLESPACE tbs_indices;
```

- Bueno para OLAP, agregaciones y filtros sobre columnas de baja cardinalidad.
- Malo para cargas OLTP muy concurrentes porque el costo de mantenimiento y bloqueo puede ser problemático.

19. Reescritura de consultas: qué suele mejorar una consulta

En esta materia no basta con crear índices: también debes reescribir SQL. Varias malas consultas se arreglan más por rediseño que por hardware.

Problema	Mala práctica	Mejor idea
Traer columnas de más	SELECT *	Seleccionar solo columnas necesarias
Eliminar duplicados sin necesidad	UNION	Usar UNION ALL si no necesitas deduplicar
Funciones en columnas filtradas	UPPER(country)=...	Normalizar dato o usar función solo si hace falta; considerar índice basado en función
Join innecesario	Unir tabla que no aporta	Eliminar join
Subconsulta costosa por fila	Subquery correlacionada mal diseñada	Evaluar join, EXISTS o agregación previa
Producto cartesiano accidental	Falta condición de join	Agregar join condition
Filtro tardío	Aplicar condiciones al final	Reducir filas lo antes posible

La diferencia entre UNION y UNION ALL es importante: UNION elimina duplicados y por eso puede exigir más trabajo (sort/unique o hashing). Si no necesitas eliminar duplicados, UNION ALL suele ser más eficiente.

20. Preguntas teóricas que probablemente te pueden formular

- Explique por qué ANSI/SPARC favorece la independencia de datos.
- Diferencie arquitectura ANSI/SPARC de arquitectura interna del motor del SGBD.
- Compare Oracle y PostgreSQL en memoria, concurrencia, recuperación y escalabilidad.
- Explique diferencias entre OLTP y OLAP con ejemplos de usuarios, consultas y modelo.
- Defina grano de una tabla de hechos y explique por qué es importante.
- Diferencie integridad, consistencia, seguridad, eficiencia y disponibilidad.
- Explique la diferencia entre usuario, rol, privilegio de sistema y privilegio sobre objeto.
- Explique la diferencia entre vista y sinónimo.
- Explique la diferencia entre join type y join method.
- Describa cómo se interpreta un plan de ejecución y qué señales sugieren mejora o problema.
- Explique cuándo un índice compuesto sí ayuda y cuándo no.
- Explique por qué un índice bitmap es más apropiado para ambientes analíticos que transaccionales.

PARTE II · GUÍA PRÁCTICA EN ORACLE SQL DEVELOPER

21. Flujo mental para resolver la parte práctica

1. Identifica con qué usuario debes ejecutar cada sentencia.
2. Si se crean usuarios u objetos administrativos, normalmente empiezas con SYSTEM o SYS.

3. Si se crea una vista, tabla o secuencia de un esquema, conéctate al dueño del objeto.
4. Después de cada GRANT, valida con una consulta al diccionario o probando el acceso real.
5. Si el ejercicio habla de rendimiento, no te quedes en la sintaxis: revisa plan, índices, filas, bytes y costos.
6. Si un script falla, depúralo por capas: orden de creación, tipos de datos, restricciones, privilegios, ruta del datafile, container, comillas y caracteres raros.

22. Plantilla base de Oracle para salas/lab

```
ALTER SESSION SET CONTAINER = XEPDB1;
```

En varios laboratorios del curso esta sentencia aparece antes de crear usuarios u objetos. Si se omite en entornos multitenant, pueden aparecer errores como ORA-65096.

23. Crear usuario, dar privilegios y asignar quota

```
CREATE USER opti IDENTIFIED BY "opti123";

ALTER USER opti
  DEFAULT TABLESPACE USERS
  TEMPORARY TABLESPACE TEMP
  ACCOUNT UNLOCK;

ALTER USER opti QUOTA 50M ON USERS;

GRANT CREATE SESSION TO opti;
GRANT CREATE TABLE TO opti;
```

Interpretación exacta: CREATE USER crea la cuenta; ALTER USER fija tablespaces y estado; QUOTA autoriza uso de espacio; GRANT CREATE SESSION permite conectarse; CREATE TABLE permite crear tablas. Si falta quota sobre el tablespace donde va a crear objetos, el usuario puede conectarse pero no necesariamente almacenar objetos allí.

24. Crear tablespace y preparar almacenamiento

```
CREATE TABLESPACE datos
DATAFILE
  'C:/Oracle/datos1.dbf' SIZE 10M AUTOEXTEND OFF,
  'C:/Oracle/datos2.dbf' SIZE 1M AUTOEXTEND OFF
SEGMENT SPACE MANAGEMENT AUTO;
```

Recuerda vocabulario: tablespace = contenedor lógico; datafile = archivo físico; segmento = espacio asignado a un objeto; extent = conjunto de bloques dentro del segmento.

- Si creas un tablespace para índices, separas físicamente objetos y mejoras administración.
- AUTOEXTEND ON ayuda a que el archivo crezca; si está OFF, una operación puede fallar por falta de espacio.
- Si el path está mal, la sentencia falla aunque el SQL sea correcto.

25. Crear roles y asignarlos a usuarios

```
CREATE ROLE tiktoky_reader;
```

```
GRANT SELECT ON users TO tiktoky_reader;
GRANT SELECT ON comments TO tiktoky_reader;

CREATE USER guest_user IDENTIFIED BY guest123;
GRANT CREATE SESSION TO guest_user;
GRANT tiktoky_reader TO guest_user;
```

Este patrón es muy importante para justificar seguridad: en vez de repetir GRANT SELECT sobre muchas tablas a cada usuario, agrupas privilegios en un rol y después asignas el rol.

Pregunta típica
¿Qué ventajas tiene usar roles? Respuesta: simplifica administración, reduce errores, centraliza cambios y favorece el principio de mínimo privilegio.

26. Crear vistas y conceder acceso

```
CREATE VIEW vw_customers AS
SELECT customer_id, name
FROM customers;

GRANT SELECT ON vw_customers TO consulta;
```

El GRANT sobre la vista debe hacerlo el dueño de la vista o alguien con autoridad suficiente. Dar acceso a la vista no da acceso automático a todas las tablas base fuera de lo que la vista expone.

27. Crear un sinónimo

```
CREATE SYNONYM customers FOR ot.vw_customers;
```

El sinónimo solo crea un alias. Si el usuario no tiene privilegio SELECT sobre la vista u objeto real, el sinónimo no “soluciona” eso.

28. Consultas con joins que sí te conviene practicar

28.1 Cliente que más ha comprado

```
SELECT c.customer_id,
       c.name AS customer_name,
       SUM(oi.quantity * oi.unit_price) AS total_spent
FROM ot.customers c
JOIN ot.orders o
  ON o.customer_id = c.customer_id
JOIN ot.order_items oi
  ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC
FETCH FIRST 1 ROW ONLY;
```

Aquí debes justificar con qué usuario corre: necesita privilegios sobre todas las tablas referenciadas. También debes saber por qué GROUP BY incluye customer_id y name: todo lo seleccionado que no es agregado debe agruparse.

28.2 Vista del cliente con mayor compra

```
CREATE VIEW vw_customers_best AS
SELECT customer_id, name
FROM (
  SELECT c.customer_id,
         c.name,
         SUM(oi.quantity * oi.unit_price) AS total_spent
  FROM customers c
  JOIN orders o
    ON o.customer_id = c.customer_id
  JOIN order_items oi
    ON oi.order_id = o.order_id
  GROUP BY c.customer_id, c.name
  ORDER BY total_spent DESC
)
FETCH FIRST 1 ROW ONLY;
```

Esta vista es útil porque encapsula lógica de negocio. Luego puedes conceder SELECT sobre ella a otro usuario.

28.3 Vista analítica con likes

```
CREATE VIEW liked_videos_stats AS
SELECT v.id AS video_id,
       v.description,
       u.username,
       u.email,
       COUNT(l.user_id) AS likes_count
FROM videos v
JOIN users u
  ON u.id = v.user_id
LEFT JOIN likes l
  ON l.video_id = v.id
GROUP BY v.id, v.description, u.username, u.email;
```

Importante: se usa LEFT JOIN para que aparezcan también videos sin likes. Si usas INNER JOIN, desaparecerían del resultado.

28.4 Top 5 videos con más likes

```
SELECT *
FROM liked_videos_stats
ORDER BY likes_count DESC
FETCH FIRST 5 ROWS ONLY;
```

29. Cómo corregir un archivo SQL con inconsistencias

Como no puedes saber de antemano cuál inconsistencia exacta tendrá el archivo del parcial, debes seguir una rutina de depuración. Esta rutina vale oro.

1. Revisa el orden: primero tablespaces/usuarios si se requieren, luego tablas, después restricciones dependientes, luego datos.
2. Verifica si la tabla usa GENERATED ALWAYS AS IDENTITY. Si es así, normalmente no insertas manualmente la PK salvo mecanismo especial.
3. Valida que las FK referencien tablas ya creadas y que los datos hijos se inserten después de los padres.

4. Busca duplicados en PK o UK.
5. Busca nulos en columnas NOT NULL.
6. Revisa comillas, comas finales, caracteres raros copiados desde PDF, guiones “–” que deberían ser “--”, y saltos de línea partidos.
7. Verifica tipos de datos: por ejemplo, insertar texto donde espera NUMBER o fechas con formato no compatible.
8. Si falla un INSERT, mira exactamente qué restricción está violando y corrige esa causa, no solo el síntoma.
9. Haz COMMIT cuando corresponda y usa bloques separados para ubicar el error más rápido.

Regla práctica
Cuando el script es largo, ejecuta por bloques pequeños: DDL primero, luego algunos inserts de prueba, luego el resto. Así detectas el primer punto real de falla.

30. Explain Plan en práctica: plantilla y lectura guiada

```
EXPLAIN PLAN FOR
SELECT student_id
FROM students
WHERE last_name = 'JOHNSON' AND first_name = 'OLIVIA';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Qué debes mirar aquí: si aparece uso del índice compuesto, si el acceso se resuelve solo con índice o si Oracle necesita ir a la tabla por ROWID, y cómo cambian rows, bytes y cost frente a otras variantes de consulta.

31. Índices en práctica: comandos esenciales

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(
    ownname => USER,
    tabname => 'STUDENTS',
    cascade => TRUE
  );
END;
/
```

Después de grandes cargas de datos conviene actualizar estadísticas. Sin esto, comparar planes puede ser engañoso.

```
CREATE INDEX idx_dni
ON students(dni)
TABLESPACE tbs_indices;
```

```
CREATE INDEX idx_students_last_first
ON students(last_name, first_name)
TABLESPACE tbs_indices;
```

```
CREATE BITMAP INDEX idx_students_gender
ON students(gender)
```

```
TABLESPACE tbs_indices;
```

32. Casos de consulta y qué esperar del plan

Consulta	Qué esperar normalmente
WHERE dni = '78901234'	Con índice en dni, buen candidato a INDEX RANGE/UNIQUE + TABLE ACCESS BY ROWID o incluso acceso muy selectivo
WHERE last_name = 'JOHNSON' AND first_name = 'OLIVIA'	Buen candidato para índice compuesto (last_name, first_name)
WHERE last_name = 'SMI%'	Puede usar índice por prefijo/rango si está bien planteado
WHERE first_name = 'OLIVIA'	No necesariamente usa bien el índice compuesto si first_name no es prefijo izquierdo
COUNT(*) con gender y city	Bitmap puede ser muy útil por baja cardinalidad y combinación de filtros

33. Reescritura de consulta DW: qué mirar

En ejercicios tipo DW/FACT_SALES suele tocar identificar grano, joins innecesarios, filtros con funciones y reescribir. Este es el checklist más útil:

- Define el grano de FACT_SALES antes de agregar.
- Elimina joins a dimensiones que no aportan columnas ni filtros.
- Evita SELECT * y selecciona solo columnas necesarias.
- Cuida filtros como UPPER(country) = SPAIN; pueden impedir mejor uso de índices si no hay índice basado en función.
- Si no necesitas eliminar duplicados, prefiere UNION ALL sobre UNION.
- Reordena la consulta para filtrar temprano y reducir filas antes de agrupar.
- Después compara el nuevo plan: costo, bytes, CPU estimada y operaciones de acceso.

34. Vistas del diccionario y de rendimiento que debes reconocer

Vista	Para qué sirve	Quién suele consultarla
DBA_USERS	Información de usuarios, tablespaces, estado	Administrador o usuario con privilegios
DBA_SYS_PRIVS	Privilegios de sistema otorgados	Administrador
DBA_TAB_PRIVS	Privilegios sobre objetos	Administrador
USER_OBJECTS / DBA_OBJECTS	Objetos del esquema o de toda la base	Usuario/DBA
USER_TABLES / DBA_TABLES	Metadatos de tablas	Usuario/DBA
USER_TAB_COLUMNS / DBA_TAB_COLUMNS	Metadatos de columnas	Usuario/DBA
DBA_TABLESPACES	Tablespaces definidos	Administrador
VSQL / VSQLAREA / V$SQLSTATS	SQL ejecutado y métricas de rendimiento	Administrador o usuario con grants especiales
V$SQL_PLAN	Plan asociado a SQL	Administrador o usuario con grants
V$FILESTAT / V$SYSSTAT	Estadísticas del sistema / archivos	Administrador

Las V$ son vistas dinámicas de rendimiento basadas en estructuras de memoria. Normalmente las consulta un DBA o un usuario al que SYS/SYSTEM le haya dado privilegios sobre V_$... o sobre las vistas correspondientes.

35. Mini banco de plantillas SQL listas para adaptar

```
-- Verificar usuario actual
SELECT USER FROM dual;

-- Verificar tablespace por defecto de un usuario
SELECT username, default_tablespace, temporary_tablespace
FROM dba_users
WHERE username = 'OT';

-- Ver privilegios de sistema
SELECT *
FROM dba_sys_privs
WHERE grantee = 'CONSULTA';

-- Ver privilegios sobre objetos
SELECT *
FROM dba_tab_privs
WHERE grantee = 'CONSULTA';
```

```
-- Consultar tabla a la que sí debería tener acceso
SELECT * FROM ot.regions;

-- Probar acceso no permitido
SELECT * FROM ot.customers;
```

```
-- Crear secuencia (solo si realmente se necesita)
CREATE SEQUENCE seq_videos START WITH 1 INCREMENT BY 1;
```

Ojo: si una columna ya es IDENTITY, crear una secuencia adicional puede ser innecesario o generar confusión.

36. Errores comunes y cómo responderlos

Error / situación	Causa probable	Qué revisar
ORA-65096	Contexto multitenant incorrecto	ALTER SESSION SET CONTAINER = XEPDB1
insufficient privileges	Falta privilegio de sistema o de objeto	Con qué usuario estás y qué GRANT falta
No space / quota exceeded	Falta quota o tablespace lleno	ALTER USER ... QUOTA ... y tamaño del datafile
Tabla no existe	Esquema incorrecto o falta sinónimo	Nombre con esquema, objeto creado, sinónimo
No funciona el sinónimo	El alias existe pero falta permiso real	GRANT SELECT sobre el objeto base
El índice no se usa	Baja selectividad, función sobre columna, mal orden de índice compuesto, pocas filas o estadísticas malas	Consulta, estadística, cardinalidad y access path
Producto cartesiano	Falta ON o condición de join	Revisar todas las relaciones del FROM

37. Checklist final antes de entregar una solución práctica

- ¿Ejecuté cada sentencia con el usuario correcto?
- ¿El usuario creado tiene CREATE SESSION y las quotas necesarias?
- ¿Los GRANT que hice son de sistema o sobre objeto, y sé justificarlo?
- ¿La vista realmente devuelve lo pedido?
- ¿El sinónimo apunta al objeto correcto?
- ¿Probé el acceso con el usuario destino?
- ¿Si la consulta era de rendimiento, revisé explain plan antes y después?
- ¿Entendí si el plan usa full scan, índice, rowid, hash join o nested loops?
- ¿La reescritura elimina trabajo innecesario y no cambia el resultado?
- ¿Mis respuestas abiertas explican el porqué y no solo copian comandos?

38. Resumen ultra concentrado para el último repaso

- ANSI/SPARC = niveles de abstracción; arquitectura del motor = componentes internos.
- OLTP = transaccional, normalizado, rápido por operación. OLAP = analítico, dimensional, histórico, agregaciones.
- Integridad no es igual a consistencia.
- Usuario se conecta; rol agrupa privilegios.
- Vista = consulta almacenada; sinónimo = alias. El sinónimo no da permisos.
- Join type ≠ join method.
- En el plan mira access path, join method, rows, bytes, cost y predicates.
- Índice compuesto depende del orden de columnas.
- Bitmap sirve mejor para baja cardinalidad y OLAP.
- Reescribir consulta casi siempre significa: menos columnas, menos joins inútiles, menos funciones en filtros, menos trabajo repetido.

39. Glosario express

Grano: nivel de detalle representado por una fila de la tabla de hechos.

Cardinalidad: cantidad de valores distintos o estimación de filas resultantes.

Selectividad: qué tanto reduce filas un predicado.

Access path: camino de acceso a datos: full scan, index scan, etc.

Join order: orden en que Oracle decide unir tablas.

Row source: flujo de filas producido por una operación del plan.

Quota: límite o permiso de uso de espacio en un tablespace.

MVCC: control de concurrencia mediante múltiples versiones de datos.

Redo / WAL: registro para recuperación tras fallos.

DBMS_XPLAN: paquete para mostrar planes de ejecución.